Д. Серікбаев атындағы Шығыс Қазақстан техникалық университеті

«DevOps құралдары» пәні

# ЗЕРТХАНАЛЫҚ ЖҰМЫС № 6, 1-бөлім

Қолданба ақаулары және қайта іске қосу

Орындаған: Есімханова Аружан Түсіпханқызы

Тобы: 25-ИСТК-2о

Оқу жылы: 2025-2026

Өскемен - 2026

МАЗМҰНЫ

1 Жұмыстың мақсаты 3

2 Теориялық мәліметтер 3

3 Жұмыстың орындалу барысы 4

4 Жұмыс нәтижелері 12

5 Талдау 12

6 Қорытынды 12

7 Пайдаланылған әдебиеттер тізімі 13

## 1 Жұмыстың мақсаты

HTTP қатесі, процесс ақауы, қайта іске қосу саясаты және штаттық тоқтату айырмашылықтарын нақты тәжірибемен анықтау.

## 2 Теориялық мәліметтер

HTTP 500 жауабы процестің аяқталғанын білдірмейді. Контейнердің негізгі процесі аяқталғанда restart саясаты қолданылады. Штаттық stop әрекеті ақаулы аяқталудан бөлек қарастырылады.

## 3 Жұмыстың орындалу барысы

ВКТУ · DEVOPS · 11-МАТЕРИАЛ

### 3.1 1-бөлім: 1-5-тармақтар, бақылау нүктесі 6

1-кесте - Орындау деректері мен тексеру нәтижелері

| Мәлімет | Мәні |
|---|---|
| Орындаған | Есімханова Аружан Түсіпханқызы |
| Тобы | 25-ИСТК-2о |
| Университет | ВКТУ |
| Күні | 2026 жылғы 23 қазан |
| Тапсырма | 11-материал, 6-зертханалық жұмыстың 1-бөлімі |

### 3.2 Мақсаты

HTTP сұрауының қатесін процестің тоқтауынан ажырату; Docker оқиғалары, exit code, JSON логтары және RestartCount арқылы төрт сценарийді дәлелдеу. restart: no мен unless-stopped саясатын салыстырып, штаттық тоқтаудың себебін түсіндіру.

### 3.3 Нақты орындалған жұмыс

Ағымдағы main коды 89a0e2b негізінде жергілікті Docker бейнесі құрастырылды. HTTP 500, автоматты қайта іске қосу, қайта іске қосуды өшіру және штаттық тоқтату нақты орындалды. Үш параллель CLI арнасы сақталды. 35 дәлел тексеруі сәтті өтті; Postman 4 сұрау мен 10 тексеруді қатесіз орындады.

Дәлелдер · Қайта орындау скрипті · Орындау тәртібі

API мекенжайы - http://localhost:8088, контейнер ішіндегі порт - 8080. Бұл компьютерде 8080 портын басқа қызмет қолданады. 12-материалдың OOM, БД ақауы және unhealthy тәжірибелері осы есепке кірмейді.

### 3.4 Дайындау және контейнерді қарау

CI бейнесіның орнына әдістеме талабына сай жергілікті main құрастыруы қолданылды: devops-labs-demo-api:lab11-89a0e2b. compose.yaml тек api.restart параметрін уақытша ауыстыру үшін өзгертілді; файлдың бастапқы байттары соңында толық қайтарылды. Атаулы PostgreSQL томы жойылған жоқ.

1-листинг - Код немесе сақталған команда шығысы

```
git switch main
git pull --ff-only
$env:API_IMAGE = "devops-labs-demo-api"
$env:API_TAG = "lab11-89a0e2b"
docker compose up -d --build --wait --wait-timeout 120
$api = "http://localhost:8088"
docker compose ps
docker compose logs api --tail 20
```

2-кесте - Орындау деректері мен тексеру нәтижелері

| Параметр | Нақты мән |
|---|---|
| API контейнері | devops-labs-api-1; running, healthy |
| Restart policy | unless-stopped |
| Лог драйвері | json-file; max-size=10m; max-file=3 |
| Ресурс шектері | Memory=268435456 байт (256 MiB); swap-пен жалпы шек 256 MiB; CPU=1 |
| ENTRYPOINT | ["dotnet", "DemoApi.dll"] - exec пішімі |
| Бастапқы RestartCount | 0 |

### 3.5 T1, T2, T3 бақылау арналары

T1 командалар мен HTTP жауаптарын жазады; T2 docker events ағынын нақты API атымен сүзеді; T3 API мен БД ресурстарын қатар өлшейді. Үш бөлек көрінетін терминал терезесінің суреті жасалмады: үш CLI арнасы параллель процестер мен файлдар арқылы тіркелді. T3 контейнер қайта жасалғанда жаңа ID-ді атауы бойынша табуы үшін --no-stream өлшемін қайталайды.

2-листинг - Код немесе сақталған команда шығысы

```
docker events --filter container=devops-labs-api-1 --filter event=start
  --filter event=kill --filter event=die --filter event=oom
  --filter event=stop --filter event=health_status
docker stats --no-stream devops-labs-api-1 devops-labs-db-1
```

T2 командасы терминалда бір жолмен енгізіледі; exec_* оқиғалары сүзілді. T3: 25 топтама, API бойынша 24 өлшем. Контейнерді қайта жасау кезінде бір stats командасы EOF/exit1 берді; қате сақталды, келесі өлшемдер жалғасты.

## 3.6 Ақау сценарийлерінің нәтижелері

3-кесте - Орындау деректері мен тексеру нәтижелері

| Сценарий | T2 оқиғалары | Exit code | Қайта іске қосу | Логтағы дерек |
|---|---|---|---|---|
| 2. Сұрау қатесі | die/start/kill/stop/oom жоқ | Процесс аяқталған жоқ | Жоқ; ID, StartedAt және RestartCount өзгермеді | Error; exception және стек |
| 3. Процестің құлауы | die(exit=1) → start → health_status: healthy | 1; die оқиғасынан алынды | Иә; сол контейнердің RestartCount мәні 0 → 1 | Critical; кейін PostgreSQL және іске қосылу логтары |
| 4. restart: no | die(exit=1); start жоқ | 1; State мәні | Жоқ; 5 с бақылаудан кейін Exited (1), RestartCount=0 | Critical; жаңа іске қосылу логы жоқ |
| 5. Штаттық тоқтату | kill(signal=15) → stop → die(exit=0) | 0; State мәні | Жоқ; қолмен start жасалғанға дейін stopped | Information: Application is shutting down... |

Кесте нақты сақталған оқиғаларға сүйенеді. Штаттық тоқтауда T2 файлы stop оқиғасын die-дан бұрын жеткізді; бұл рет әдейі өзгертілмеді. Негізгі белгілер - SIGTERM=15, exit=0, stop және автоматты start болмауы.

### 3.7 Exit code пен HTTP коды екі бөлек өлшем

HTTP 500 бір сұраудың нәтижесін білдіреді. Ол контейнер процесінің exit code-ы емес. 2-сценарийде процесс тірі болғандықтан, оның аяқталу коды жоқ. 3-сценарийде қайта іске қосылған контейнердің ағымдағы State.ExitCode мәні 0 болуы мүмкін; бұрынғы 1 коды die оқиғасында сақталады.

### 3.8 Дәлел сілтемелері

2: сұрау қатесі · 3: автоматты қайта іске қосу

4: no саясаты · 5: штаттық тоқтату

Әр JSON файлында UTC аралығы, контейнердің before/after күйі, HTTP жауабы, лог жазбалары және сол уақыт аралығындағы нақты оқиғалар бар. Бөлек толық T2 ағыны Compose қайта жасаған кездегі қосымша оқиғаларды да сақтайды.

### 3.9 Сұрау қатесі және процестің авариялық аяқталуы

### 3.10 2-сценарий: HTTP 500, процесс тірі

3-листинг - Код немесе сақталған команда шығысы

```
curl.exe -s -w " [%{http_code}]`n" "$api/api/chaos/error"
docker compose logs api --tail 3 --no-log-prefix
```

4-листинг - Код немесе сақталған команда шығысы

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.6.1",
  "title": "An error occurred while processing your request.",
  "status": 500,
  "traceId": "00-11be4563175ace3df3d4f1b6cb7cd9e7-7d84eaf95437df08-00"
}
```

Жауап application/problem+json пішімінде, status=500. Ішкі Chaos хабарламасы мен exception stack trace клиентке шыққан жоқ. Олар сервер логында Error деңгейімен сақталған:

5-листинг - Код немесе сақталған команда шығысы

```
LogLevel: Error
Category: Microsoft.AspNetCore.Diagnostics.ExceptionHandlerMiddleware
Exception: System.InvalidOperationException: Chaos: искусственная ошибка
  at DemoApi.Controllers.ChaosController.Error()
```

Толық стек бөлек лог файлында берілген; бұл қысқа үзінді негізгі белгілерін ғана көрсетеді. API контейнерінің ID-і, StartedAt және RestartCount өзгермеді; /health/live 200 қайтарды.

### 3.11 3-сценарий: exit 1 және автоматты қайта іске қосу

6-листинг - Код немесе сақталған команда шығысы

```
curl.exe -X POST "$api/api/chaos/crash"
docker inspect devops-labs-api-1 --format "{{.RestartCount}}"
docker compose logs api --since 2m --no-log-prefix
```

POST /api/chaos/crash алдымен 202 қайтарады; шамамен 200 мс кейін қолданба Environment.Exit(1) шақырады. Critical логы жазылып, процесс тоқтайды. unless-stopped саясаты сол контейнерді қайта іске қосады. T2 тізбегі:

7-листинг - Код немесе сақталған команда шығысы

```
die (exit=1) → start → health_status: healthy
```

RestartCount 0-ден 1-ге өсті, контейнер ID-і өзгермеді, StartedAt жаңарды. Readiness 200-ге оралды; бақылау жобасы мен тапсырмасы PostgreSQL-де сақталды. Бұл Compose арқылы контейнерді жаңадан жасаудан бөлек жағдай.

### 3.12 no саясаты және штаттық тоқтату

### 3.13 4-сценарий: автоматты қайта іске қосуды өшіру

compose.yaml ішінде тек api қызметінің restart мәні уақытша "no" деп өзгертілді. DB мен Adminer саясаттары сол қалпында қалды. Compose up --no-build --wait контейнерді жаңа конфигурациямен қайта жасады; содан кейін crash сұрауы жіберілді.

8-листинг - Код немесе сақталған команда шығысы

```
restart: "no"
docker compose up -d --no-build --wait --wait-timeout 90
curl.exe -X POST "$api/api/chaos/crash"
Start-Sleep 5
docker compose ps -a
```

4-кесте - Орындау деректері мен тексеру нәтижелері

| Өлшем | Нәтиже |
|---|---|
| State.Status | exited |
| State.ExitCode | 1 |
| RestartCount | 0 |
| Оқиға | die (exit=1) |
| Лог | Critical: Chaos: аварийное завершение процесса |

Параметр unless-stopped мәніне қайтарылып, API қайта іске қосылды. Бақылау деректері сақталды. Compose файлының бастапқы және соңғы SHA-256 мәндері бірдей; уақытша no параметрі Git-ке енгізілген жоқ.

## 3.14 5-сценарий: SIGTERM арқылы штаттық тоқтату

9-листинг - Код немесе сақталған команда шығысы

```
docker compose stop api
docker compose logs api --tail 5 --no-log-prefix
docker compose start api
```

Оқиғалар: kill (signal=15) → stop → die (exit=0). Лог: Application is shutting down...; State.ExitCode=0. SIGKILL=9 тіркелген жоқ. Контейнер қолмен start жасалғанға дейін 5 секундтық бақылауда stopped күйінде қалды.

docker compose start api командасынан кейін API healthy болды, live/ready 200 қайтарды. Кері қайтару немесе down/up қолданылған жоқ; дәлел ретінде бастапқы ақау логтары мен оқиғалары сақталды.

## 3.15 Бақылау сұрақтарына жауап

### 3.16 2-сценарий 3-сценарийден пайдаланушы және Docker үшін несімен ерекшеленеді?

5-кесте - Орындау деректері мен тексеру нәтижелері

| Өлшем | 2: сұрау қатесі | 3: процесс тоқтайды |
|---|---|---|
| Пайдаланушыға әсері | Бір сұрау HTTP 500 қайтарады; басқа сұраулар орындала береді | Crash сұрауы 202 алуы мүмкін; қайта іске қосылу кезінде жаңа сұраулар үзіледі не қосылу қатесін алады |
| Docker күйі | Процесс running/healthy; қайта іске қосу жоқ | Процесс exit 1 береді; policy рұқсат етсе, Docker қайта іске қосады |
| Негізгі дәлел | Error және exception stack; lifecycle оқиғасы жоқ | Critical, die(exit=1), start, RestartCount өсімі |

Осылайша, HTTP 500 саны мен контейнердің авариялық аяқталу саны бірдей емес. Тек docker compose ps-ке қарап 2-сценарийді анықтау мүмкін емес: контейнер healthy көрінеді. Сұрау логтары мен HTTP нәтижесі қажет.

### 3.17 docker compose stop api орындалғанда unless-stopped неге қайта іске қоспайды?

Бұл - әкімшінің әдейі тоқтатуы. unless-stopped саясаты мұндай контейнерді автоматты көтермейді; әйтпесе оны қолмен өшіру мүмкін болмас еді. Контейнерді қайта қосу үшін docker compose start api сияқты нақты команда қажет. Docker daemon қайта іске қосылса да unless-stopped қолмен тоқтатылған контейнерді көтермейді; always саясатының осы жағдайда айырмасы бар.

Процестің өзі қателікпен аяқталғанда қолмен тоқтату белгісі қойылмайды: 3-сценарийде сондықтан автоматты қайта іске қосу болды. 5-сценарийде SIGTERM-ді ASP.NET өңдеп, қолданбаны exit 0 арқылы аяқтады; бұл error crash емес.

### 3.18 Ресми анықтамалар

Docker restart policies · Docker stop және сигналдар · Docker events

Бұлар механизмді түсіндіретін ресми дереккөздер; кестедегі нақты нәтижелер репозиторийдегі өлшенген JSON/лог файлдарынан алынған.

### 3.19 Қорытынды нәтижелер

6-кесте - Орындау деректері мен тексеру нәтижелері

| Талап | Күйі |
|---|---|
| 1: бастапқы қарау | ps, logs, LogConfig, restart policy, ресурстар және ENTRYPOINT сақталды |
| 2-5: төрт сценарий | HTTP, events, exit code, logs және restart нақты тексерілді |
| Сценарийлер кестесі | Сценарийлер кестесінде бес бағанмен берілген |
| Сұрақтар 1 және 2 | Бақылау сұрақтары бөлімінде жауап берілген |
| Дәлел тексерулері | 35 сәтті, қате 0 |
| Postman | 4 сұрау, 10 assertion, қате 0; нақты localhost API |
| Compose және деректер | Бастапқы файл толық қайтарылды; атаулы том, жоба және тапсырма сақталды |

Қазақша есептер мен ортақ мұрағат · 35 тексеру

### 3.20 Қауіпсіздік

Тәжірибелер тек localhost-тағы оқу Compose жобасында жасалды. DB_PASSWORD пен GitHub токендері есепке қосылған жоқ. Chaos crash сұрауы негізгі Postman коллекциясына енгізілмеді; оны тек айқын таңдалған тексеру скрипті орындайды. Құпия деректер көрінуі мүмкін толық inspect Config.Env жарияланбады.

## 4 Жұмыс нәтижелері

Төрт сценарий және 35 тексеру, T1/T2/T3 уақыттары, Postman 4 сұрау/10 тексеру сақталған. Уақытша restart: no конфигурациясы қайтарылған.

Тест нәтижелері мен журналдар жұмыс материалдарында [1], іске қосу командалары нұсқаулықта [2] берілген.

## 5 Талдау

HTTP 500 жеке сұрауға әсер етті, бірақ процесті тоқтатпады. Exit 1 негізгі процесті аяқтап, restart саясатын іске қосты. Қолмен stop кезінде unless-stopped контейнерді өздігінен қайта қоспады.

## 6 Қорытынды

Сұрау қатесі, процесс ақауы және қалыпты тоқтау салыстырылды. Төрт сценарий бойынша 35 тексеру өтті.

## 7 Пайдаланылған әдебиеттер тізімі

[1] DemoApi оқу репозиторийі. Нақты тексерулер мен сақталған дәлелдер. Электрондық ресурс. https://github.com/Aruzhaaan-06/devops-labs/blob/main/evidence/lab06-failures/README.md. Қаралған күні: 25.10.2026.

[2] DemoApi оқу репозиторийі. Жұмыстың орындалу тәртібі. Электрондық ресурс. https://github.com/Aruzhaaan-06/devops-labs/blob/main/docs/lab06-failures.md. Қаралған күні: 25.10.2026.

[3] Docker restart policies. Электрондық ресурс. https://docs.docker.com/engine/containers/start-containers-automatically/. Қаралған күні: 25.10.2026.

[4] Docker stop және сигналдар. Электрондық ресурс. https://docs.docker.com/reference/cli/docker/container/stop/. Қаралған күні: 25.10.2026.

[5] Docker events. Электрондық ресурс. https://docs.docker.com/reference/cli/docker/system/events/. Қаралған күні: 25.10.2026.

DevOps пәнінің оқу материалдары: Devops (1).zip мұрағаты, зертханалық тапсырмалар және бақылау нүктелерінің шарттары.

ГОСТ 7.32-2017. Ғылыми-зерттеу жұмысы туралы есеп. Құрылымы және рәсімдеу ережелері. 6-бөлім.